

Activity 03: Trunks and inter-VLAN routing

TGS-2023037854 | v7.0 | 13 September 2026 | 100 minutes

Goal

Diagnose a missing allowed VLAN and native VLAN mismatch, then verify SVI gateway mapping.

Before you start

Python 3.9+, a plain text editor and this complete folder. Offline mode requires no extra packages, Internet or credentials. The compulsory core Packet Tracer construction is specified below and in topology.md; advanced extensions may need CML/IOS XE.

Evidence boundary

All JSON/CSV observations are synthetic training evidence. Passing the Python checker validates the fixture contracts only. No hardware, simulator, authentication server or API is configured or contacted.

Files

README.md, instructions.pdf, checklist.pdf, topology.md, case.json, faulty.json, reference.json, observations.csv, specimen.cfg, D1-baseline.cfg, S1-baseline.cfg, S2-baseline.cfg

Working method

case.json is the working evidence; faulty.json resets the exercise and reference.json shows corrected synthetic evidence for self-study. Preserve your own sandbox observations separately.

Detailed step-by-step instructions

1. Construct and save the mandatory simulator topology

Start Cisco Packet Tracer 8.x or a current compatible version. File > New; use the bottom device palette, drag these models into the workspace and rename them via Config > Display Name: 1 x Multilayer Switch 3560-24PS (D1), 2 x Switch 2960 (S1,S2), 2 x PC-PT (PC1,PC2). Choose Connections > Copper Straight-Through for the table unless otherwise stated. Switch-to-switch links use Copper Cross-Over or Automatically Choose Connection Type. Click the first device and select its named interface; click the peer and select its named interface. Wait for links to initialize. Save as activity-start.pkt.

Device/port A	Device/port B	Cable
PC1 FastEthernet0	S1 FastEthernet0/1	Straight-through
PC2 FastEthernet0	S2 FastEthernet0/1	Straight-through
S1 GigabitEthernet0/1	D1 GigabitEthernet0/1	Cross-over/Auto
S2 GigabitEthernet0/1	D1 GigabitEthernet0/2	Cross-over/Auto

Save both activity-start.pkt and a working activity-working.pkt; use File > Save As.

2. Assign endpoint addresses and inspect interface identifiers

Set Desktop > IP Configuration > Static on each endpoint.

Endpoint	Address	Mask	Gateway	DNS
PC1	10.10.10.10	255.255.255.0	10.10.10.1	none
PC2	10.10.20.20	255.255.255.0	10.10.20.1	none

For routers/switches open CLI. Answer no to the initial configuration dialog and press Enter. Use the exact interface names below; a 2911 uses GigabitEthernet0/0 through 0/2, a 2960/3560 uses FastEthernet0/1 through 0/24 and GigabitEthernet0/1 through 0/2.

```
enable
show ip interface brief
```

3. Install all peer baseline configurations

Paste the following device-specific blocks into each named device CLI one device at a time. Wait for the router/switch prompt between blocks. These are complete activity configurations for the stated construction. Do not apply configuration from a different activity. When copy asks for Destination filename, press Enter. Save the Packet Tracer working file after all blocks.

Device D1

```
enable
configure terminal
hostname D1
no ip domain-lookup
vlan 10
  name USERS
vlan 20
  name SERVICES
vlan 99
  name NATIVE
interface range GigabitEthernet0/1-2
  switchport trunk encapsulation dot1q
  switchport mode trunk
  switchport trunk native vlan 99
  switchport trunk allowed vlan 10,20,99
```

```
no shutdown
exit
ip routing
interface Vlan10
 ip address 10.10.10.1 255.255.255.0
 no shutdown
exit
interface Vlan20
 ip address 10.10.20.1 255.255.255.0
 no shutdown
exit
end
copy running-config startup-config
```

Device S1

```
enable
configure terminal
hostname S1
no ip domain-lookup
vlan 10
 name USERS
vlan 20
 name SERVICES
vlan 99
 name NATIVE
interface FastEthernet0/1
 switchport mode access
 switchport access vlan 10
 no shutdown
exit
interface GigabitEthernet0/1
 switchport mode trunk
 switchport trunk native vlan 99
 switchport trunk allowed vlan 10,20,99
 no shutdown
exit
end
copy running-config startup-config
```

Device S2

```
enable
configure terminal
hostname S2
no ip domain-lookup
vlan 10
 name USERS
vlan 20
 name SERVICES
vlan 99
 name NATIVE
interface FastEthernet0/1
 switchport mode access
 switchport access vlan 20
 no shutdown
exit
interface GigabitEthernet0/1
 switchport mode trunk
 switchport trunk native vlan 99
 switchport trunk allowed vlan 10,20,99
 no shutdown
exit
end
copy running-config startup-config

show running-config
show ip interface brief
copy running-config startup-config
```

4. Prove the healthy baseline with actual packets

Run every test below before introducing the fault. PC/Desktop > Command Prompt provides ping, ipconfig and arp. Use Simulation mode and Edit Filters to keep ARP, ICMP and the protocol under study; click Add Simple PDU for a ping and Capture/Forward to inspect hops. The first ping may lose one packet during ARP; repeat and record the second result. Do not mark a test passed until observed.

PC1 ping 10.10.10.1; PC2 ping 10.10.20.1; PC1 ping 10.10.20.20; PC2 ping 10.10.10.10. All succeed before/after repair. D1 show ip route connected displays C 10.10.10.0/24 and C 10.10.20.0/24. show ip interface brief shows Vlan10/Vlan20 up/up. On all switches show interfaces trunk displays native99 and VLAN10,20,99 allowed.

After all healthy tests pass, File > Save As > activity-start.pkt to preserve the CONFIGURED healthy baseline, then File > Save As > activity-working.pkt before introducing the fault. The initial empty construction is not the rollback baseline.

Save baseline screenshots / CLI text with device names; File > Save.

5. Introduce and diagnose the controlled fault

Save the baseline first. Apply only the following fault to the specified device. Repeat the same traffic tests, capture the first failing hop/control and compare show output with the healthy baseline.

On S2 remove VLAN20 from the uplink:

```
enable
configure terminal
interface GigabitEthernet0/1
  switchport trunk allowed vlan 10,99
end
```

PC2 ping 10.10.20.1 fails; PC1 ping 10.10.10.1 remains healthy. Capture show interfaces trunk on S2 and D1. As a separate fault set S2 native VLAN 1; inspect the native VLAN mismatch and then restore 99.

Retain activity-start.pkt and baseline running configurations; work in activity-working.pkt.

6. Apply the narrow repair and re-test the network

On S2: configure terminal; interface GigabitEthernet0/1; switchport trunk native vlan 99; switchport trunk allowed vlan 10,20,99; end. Inspect both ends, not just D1.

Repeat the baseline test matrix. Check surrounding controls as well as the repaired flow. how the trunk preserves VLAN tags while D1 routes between its SVIs; explain why a VLAN must exist and have an active forwarding port for the SVI to be up

Save actual show output and allowed/denied traffic results; copy running-config startup-config only after successful verification.

7. Rollback and rehearse recovery

Save the repaired file as activity-verified.pkt. To undo the exercise, open the saved healthy baseline activity-start.pkt after you have saved configurations into it at Step 3, or paste the saved pre-fault running configuration into a clean topology. For the specific repair, retain the exact old command/value rather than guessing a trunk list or ACL. Repeat one positive and one relevant negative test after recovery.

File > Save As > activity-verified.pkt; close working topology; reopen saved healthy baseline.

8. Submit observed evidence and explain the forwarding mechanism

Submit the verified .pkt file, a cabling/address table, pre-fault and post-repair show outputs, screenshots of traffic tests, diagnosis and rollback notes. Explain how the trunk preserves VLAN tags while D1 routes between its SVIs; explain why a VLAN must exist and have an active forwarding port for the SVI to be up. Keep synthetic checker output separate from real simulator observations.

Record simulator version, device models and tests actually completed.

Untimed appendix: independent fixture analysis

These synthetic cases supplement the allocated hands-on time and do not configure the simulator.

Appendix 1. Establish the baseline

Open activity03-trunks-inter-vlan-routing; inspect topology.md, case.json and observations.csv. Identify the controls for: Diagnose a missing allowed VLAN and native VLAN mismatch, then verify SVI gateway mapping.

```
python3 --version
```

Appendix 2. Run the faulty evidence case

Run the checker against case.json; exit status 1 is expected at this stage. Record each FAIL label and the calculated detail.

```
python3 checker.py case.json
```

Appendix 3. Propose the repair

Set the right trunk native VLAN to 99 and allowed VLANs to [10,20,99]. Keep VLAN 99 distinct from user VLANs. Validate the remote trunk before changing an SVI.

Open case.json in a plain text editor; preserve JSON types.

Appendix 4. Validate the corrected case

Save a separate working copy first. Run the checker after each narrow edit until all controls PASS; capture output as evidence. This validates fixtures only.

```
python3 checker.py case.json > evidence.txt
```

Appendix 5. Verify checker behavior

Run self-test; it must accept corrected reference evidence and reject the faulty fixture and the independent corrupted-control test.

```
python3 checker.py --self-test
```

Expected results and evidence

Corrected reference: all checks PASS, exit 0; faulty fixture: one or more FAIL, exit 1; self-test: PASS, exit 0. Domain-specific computed fields appear in JSON detail.

Capture evidence.txt, corrected case.json and your diagnosis. For sandbox work record device/model, timestamps, actual show outputs, allowed/denied tests and rollback. Distinguish fixture results from real observations.

Troubleshooting and reset

Exit 1: inspect the named failed contract. Exit 2: restore valid JSON keys/types. Unsupported simulator commands must be documented; never invent results. Reset by copying faulty.json over case.json. Restore sandbox settings from the actual saved baseline.

Calculated corrected-fixture results

common_allowed_vlans: [10, 20, 99]

gateways: ["10.10.10.1/24", "10.10.20.1/24"]

Required PASS controls

- both interfaces trunks
- native VLAN agreement
- end-to-end VLAN allowance
- layer3 enabled
- SVI gateways

Configuration files

The complete per-device configurations are shown in the construction steps and included as named *-baseline.cfg files. specimen.cfg combines those same blocks with explicit device boundaries. Apply each block only to its named device. WRT300N uses GUI settings instead of IOS CLI.

References

Cisco CCNA v1.1 exam topics; Cisco IOS XE configuration guides; Python ipaddress documentation. Exact URLs and source links are in README.md.